Full Stack Developer Assignment – Backtesting Framework for Equity-Based Strategies

Objective

Build an end-to-end backtesting platform for equity-based strategies that combines Python-based backend logic, a structured database, and a React-based frontend. The user must fetch data using (yahoo finance or other data providers) and allow users to define and test custom fundamental strategies through an intuitive UI.

1. Backtest Engine (Python)

The backtest logic must support the following features:

- User-defined start and end dates
- Rebalancing frequency: monthly, quarterly, yearly, etc.
- Portfolio size: e.g., top 20 stocks
- Position sizing methods:
  - Equal-weighted
  - Market cap-weighted
  - Metric-weighted (e.g., ROCE)
- Filtering system (applied once at the beginning, used for every rebalance):
  - Market Cap range (e.g., ₹1000 Cr to ₹10000 Cr)
  - ROCE > X%
  - PAT > 0
- Ranking system:
  - Sort based on single or multiple metrics (e.g., ROE descending, PE ascending)
  - Support composite ranking by averaging individual ranks
- Initial capital input and compounding logic at each rebalance
- Ensure no future data leakage

2. Data Collection (Web Scraping / API Only)

The candidate must fetch data using APIs or web scraping. The collected dataset must include:

- At least 100 Indian listed companies
- Historical stock price data (OHLCV)
- Fundamental data: P&L, Balance Sheet, Cash Flow items and some performance & valuation ratios/metrics.

Use sources like Yahoo Finance, Screener, Ticker, or other relevant APIs. Ensure data is clean and structured before storing it in the database.

3. Database (PostgreSQL or MySQL)

You must design and use a relational database to store all price and fundamental data.

- Normalize and index tables for fast access
- Separate tables for stock prices and financial metrics
- Store all scraped or API-fetched data here before running any backtest

4. Frontend (React or Next.js + Tailwind CSS)

The frontend must allow users to:

- Configure backtest parameters: date range, rebalance frequency, portfolio size
- Input stock filters and ranking logic
- Select position sizing method
- Run the backtest and view outputs

Outputs to be displayed:

- Equity curve
- Drawdown chart
- Performance metrics: CAGR, Sharpe, Max Drawdown, etc.
- Top winners and losers
- Portfolio logs with weights and returns
- Option to export results as CSV or Excel

5. Submission Requirements

Deliverable	Details
GitHub Repository	All source code, with a clean directory structure
README File	Setup steps, architecture overview, how to run frontend and backend
Video Demo	3–10 minute screen recording showing the app workflow and features
Documentation	Describe each module, file structure, assumptions, and optional features

6. Optional Features (Bonus)

- API endpoints for backtesting via Flask or FastAPI
- Outputs of prebuilt stratgies
- Strategy comparison feature
- Comparison of the strategy outputs with benchmark (Nifty 50)

7. Timeline

The project must be completed and submitted within **2 days** from the date of assignment.

8. Tech Stack Summary

- **Backend:** Python (pandas, numpy, SQLAlchemy)
- **Database:** PostgreSQL or MySQL
- **Frontend:** React.js or Next.js + Tailwind CSS
- **Data Sources:** Yahoo Finance, Screener, APIs, or custom scraping

9. Evaluation Criteria

- Accuracy and modularity of backend logic
- Database schema design and efficiency
- Frontend responsiveness and usability
- Depth of metrics and analytics presented
- Code quality, documentation, and delivery presentation

This assignment reflects a real-world project you'd work on at Qode. We're looking for candidates who can think end-to-end — from clean data ingestion and scalable backend to intuitive frontend delivery.